

Final Team Project Report (TP406)

Project Description

Problem and Its Analysis

Our team's central topic is **Problem 5 (Low-Rank Tensor Approximation)**. Given an arbitrary tensor $T \in \mathbb{C}^3 \otimes \mathbb{C}^3 \otimes \mathbb{C}^3$ — the "Rubik's cube" case — the goal is to find the best rank-2 approximation

$$T' \in \sigma_2(\text{Seg}(\mathbb{C}^3 \times \mathbb{C}^3 \times \mathbb{C}^3))$$

minimizing the distance $\|T - T'\|$, together with a numerical implementation.

Two preliminary choices fix the problem precisely. **(i) The distance function.** We adopt the Frobenius norm $\|\cdot\|_F$, the tensor analogue of the Euclidean norm. Because it is induced by an inner product it is differentiable, which turns the approximation into a tractable least-squares problem. **(ii) The geometry.** The target set σ_2 is the *closure* of the rank- ≤ 2 tensors. This is not a mere technicality: the set of tensors of rank exactly ≤ 2 is not closed, so a sequence of rank-2 tensors may converge to a tensor of higher rank (the *border-rank* phenomenon). A best rank-2 approximation can therefore fail to exist on the open set, whereas passing to the closure σ_2 guarantees a minimizer exists. We keep this in mind when an iteration drifts toward degenerate (diverging) factors, though a deeper study of border rank is beyond the scope of this project.

Solution Strategy

A rank- R CP (Canonical Polyadic) decomposition writes a tensor as a sum of R rank-one tensors, so a rank-2 approximation is exactly a CP decomposition with $R = 2$. This makes the **CP decomposition** the natural model for Problem 5, and we solve it with the **Alternating Least Squares (ALS)** algorithm, which minimizes the Frobenius-norm objective by updating one factor matrix at a time. We treat the L2 (Frobenius) objective as primary and additionally examine an L1 objective to gauge robustness against impulsive noise. We verify the implementation by tracking the **relative error** on controlled cases — exact rank-2 recovery, noisy rank-2 recovery, and generic random tensors — and by cross-checking the result against an established library.

Beyond Problem 5: Tucker Decomposition

To go further, we introduce the **Tucker decomposition**, a more general factorization in which CP is a special case (a superdiagonal core). Tucker assigns a separate rank to each mode — the *multilinear rank* — giving more flexibility than CP's single rank R . We implement two standard algorithms: **HOSVD**, a closed-form method that applies the SVD mode by mode, and **HOOI**, which refines the HOSVD result iteratively toward a better fit.

Evaluation Metrics

We use three metrics with distinct roles. The **relative error** $\varepsilon_{\text{rel}} = \|X - \hat{X}\|_F / \|X\|_F$ validates the correctness and accuracy of the decomposition algorithms themselves. **PSNR** and **SSIM** assess reconstruction quality in the image application: PSNR measures pixel-level fidelity, while SSIM captures perceived structural similarity.

Application: Image Inpainting

Finally, we apply all three algorithms (CP-ALS, HOSVD, HOOI) to **image inpainting**. An RGB image is naturally a third-order tensor $I \in \mathbb{R}^{H \times W \times 3}$. We simulate corruption with a random mask (a set of missing pixels) and exploit the low-rank structure of the image tensor to reconstruct the missing regions. Comparing PSNR and SSIM across algorithms and ranks shows how effectively each method recovers a masked image, and reveals that the rank is the dominant factor governing the quality-compression trade-off.

Project Details — Theoretical Background

1. Frobenius Norm

The Frobenius norm of a tensor $X \in \mathbb{R}^{I_1 \times \dots \times I_N}$ is

$$\|X\|_F = \sqrt{\langle X, X \rangle} = \sqrt{\sum_{i_1=1}^{I_1} \dots \sum_{i_N=1}^{I_N} x_{i_1 \dots i_N}^2},$$

the square root of the sum of its squared entries. This is the distance function we minimize, so it *defines* what "best approximation" means in Problem 5. Being induced by the tensor inner product (Appendix A.1), it is differentiable — which is precisely what lets every approximation problem below be cast as a least-squares problem.

2. Matricization (Unfolding or Flattening)

The mode- n matricization $X_{(n)}$ rearranges the mode- n fibers of a tensor into the columns of a matrix:

$$\mathbb{R}^{I_1 \times \dots \times I_N} \longrightarrow \mathbb{R}^{I_n \times (I_1 \dots I_{n-1} I_{n+1} \dots I_N)}.$$

This is the conceptual bridge of the entire project: it converts multilinear tensor operations into ordinary matrix algebra, making least squares, the SVD, and the pseudo-inverse available. Both ALS and HOSVD are defined on these unfoldings. The explicit index map and a verification example are given in Appendix A.2.

3. CP Decomposition

For a third-order tensor $X \in \mathbb{R}^{I \times J \times K}$, the CP decomposition is

$$X \approx \sum_{r=1}^R a_r \circ b_r \circ c_r, \quad x_{ijk} = \sum_{r=1}^R a_{ir} b_{jr} c_{kr}.$$

Collecting the component vectors into factor matrices $A = [a_1 \dots a_R]$, B , and C , it admits the matricized form

$$X_{(1)} \approx A(C \odot B)^\top, \quad X_{(2)} \approx B(C \odot A)^\top, \quad X_{(3)} \approx C(B \odot A)^\top,$$

where $\odot$ is the Khatri–Rao product (Appendix A.3). The first identity encodes the model for Problem 5 — the rank R is the number of rank-one terms — while the matricized form makes the model *computable*, since it is linear in each factor matrix.

4. Alternating Least Squares (ALS)

ALS solves

$$\min_{A, B, C} \left\| X - \sum_{r=1}^R \lambda_r a_r \circ b_r \circ c_r \right\|_F,$$

which is non-convex in A, B, C jointly but becomes a *linear least-squares* problem once all factors but one are fixed. Fixing B and C and using the matricized form yields the closed-form update

$$A' \leftarrow X_{(1)}(C \odot B)(C^\top C * B^\top B)^\dagger,$$

where $*$ is the Hadamard product; we then set $\lambda_r = \|a'_r\|$ and normalize each column. Cycling this update over all modes until the fit stops improving is the algorithm (a block coordinate descent). Its importance is twofold: it reduces an intractable joint optimization to a sequence of solvable subproblems, and the identity $(C \odot B)^\top (C \odot B) =$

$C^\top C * B^\top B$ lets each step avoid forming the large $IJ \times R$ matrix, keeping it efficient. The full derivation and pseudocode are in Appendix A.4.

We also considered three alternative CP algorithms but did not adopt them: **ASD** avoids matricization but is mathematically less accurate than ALS; **dGN** (damped Gauss–Newton) and the **PMF3** method built on it incur high memory and computational cost from Hessian-based large-scale operations. ALS offers the best accuracy–cost balance for our setting.

5. Tucker Decomposition

The Tucker decomposition factorizes $X \in \mathbb{R}^{I \times J \times K}$ as

$$X \approx G \times_1 A \times_2 B \times_3 C,$$

where $G \in \mathbb{R}^{P \times Q \times R}$ is the *core tensor* and A, B, C are the factor matrices; $\times_n$ denotes the n -mode product (Appendix A.5). When P, Q, R are smaller than I, J, K , the core G is a compressed version of X . The triple of mode ranks (R_1, R_2, R_3) , with $R_n = \text{rank}(X_{(n)})$, is the **multilinear rank**. CP is the special case in which the core is superdiagonal and $P = Q = R$; conversely, allowing a separate rank per mode makes Tucker strictly more flexible than CP, which is why it can reach higher reconstruction quality at a comparable compression ratio.

6. HOSVD and HOOI

Just as the SVD underlies PCA for matrices, HOSVD and HOOI are its tensor counterparts (the SVD background is in Appendix A.1 and A.6). **HOSVD** sets each factor $A^{(n)}$ to the R_n leading left singular vectors of the unfolding $X_{(n)}$ and forms the core by projection. It is closed-form and non-iterative, but the *truncated* HOSVD ($R_n < \text{rank}_n X$) is not the optimal fit. **HOOI** closes this gap by iterating: starting from the HOSVD solution, it refines the factors until convergence. The key fact (derived in Appendix A.7) is that for columnwise-orthonormal factors,

$$\|X - \llbracket G; A^{(1)}, \dots, A^{(N)} \rrbracket\|^2 = \|X\|^2 - \|G\|^2,$$

so minimizing the error is equivalent to *maximizing* $\|G\|$, which reduces to a single SVD per mode. HOSVD thus supplies a fast initialization and HOOI supplies the (locally) optimal refinement.

Appendix

Kolda, T. G., & Bader, B. W. (2009). Tensor decompositions and applications. SIAM review, 51(3), 455-500.

Paatero, P. (1997). A weighted non-negative least squares algorithm for three-way ‘PARAFAC’ factor analysis. Chemometrics and Intelligent Laboratory Systems, 38(2), 223-242.

Appendix

A.1 Tensor Inner Product, Frobenius Norm, and the SVD

For $X, Y \in \mathbb{R}^{I_1 \times \dots \times I_N}$,

$$\langle X, Y \rangle = \sum_{i_1=1}^{I_1} \dots \sum_{i_N=1}^{I_N} x_{i_1 \dots i_N} y_{i_1 \dots i_N}, \quad \|X\|_F = \sqrt{\langle X, X \rangle}.$$

The inner product underlies every orthogonality and norm argument used below.

For a matrix $A \in \mathbb{R}^{m \times n}$, the SVD is $A = USV^\top$ with U, V columnwise orthonormal and S diagonal. From

$$A^\top A = VS^\top U^\top USV^\top = V(S^\top S)V^\top \implies A^\top A V = \sigma^2 V,$$

the columns of V are eigenvectors of $A^\top A$ and those of U are eigenvectors of $AA^\top$, with the σ_i as singular values. HOSVD and HOOI apply this construction to each unfolding.

```
def inner_product(X, Y):
    return np.sum(X * Y)

def frobenius_norm(X):
    return np.sqrt(inner_product(X, X))
```

A.2 Matricization: Index Map

The mode- n matricization maps a tensor index $(i_1, \dots, i_N)$ to a matrix position (i_n, j) with

$$j = 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) J_k, \quad J_k = \prod_{\substack{m=1 \\ m \neq n}}^{k-1} I_m.$$

Example ($X \in \mathbb{R}^{3 \times 4 \times 2}$): the entry $x_{2,3,2}$ maps to column

$$j = 1 + (3 - 1) \cdot 1 + (2 - 1) \cdot 4 = 7,$$

i.e. position $(2, 7)$ of $X_{(1)}$, which agrees with the direct unfolding.

A.3 Khatri–Rao Product

For $A \in \mathbb{R}^{I \times R}$ and $B \in \mathbb{R}^{J \times R}$, the Khatri–Rao product is the columnwise Kronecker product

$$A \odot B = [a_1 \otimes b_1 \quad a_2 \otimes b_2 \quad \dots \quad a_R \otimes b_R] \in \mathbb{R}^{IJ \times R}.$$

The identity that powers ALS is

$$(A \odot B)^\top (A \odot B) = (A^\top A) * (B^\top B),$$

where $*$ is the Hadamard (elementwise) product. It replaces an $IJ \times R$ computation with an $R \times R$ one.

A.4 CP-ALS: Derivation and Pseudocode

Fixing B and C , the mode-1 subproblem is $\min_{A'} \|X_{(1)} - A'(C \odot B)^\top\|_F$, whose least-squares solution uses the pseudo-inverse:

$$A' = X_{(1)} [(C \odot B)^\top]^\dagger = X_{(1)} [(C \odot B)^\dagger]^\top.$$

Using $M^\dagger = (M^\top M)^\dagger M^\top$ together with the identity of A.3,

$$(C \odot B)^\dagger = (C^\top C * B^\top B)^\dagger (C \odot B)^\top \implies A' = X_{(1)} (C \odot B) (C^\top C * B^\top B)^\dagger.$$

Normalizing each column gives $\lambda_r = \|a'_r\|$ and $a_r = a'_r / \lambda_r$. The mode-2 and mode-3 updates are symmetric.

```
procedure CP-ALS(X, R)
    initialize A^(n) in R^{I_n \times R} for n = 1, ..., N
    repeat
        for n = 1, ..., N do
            V <- A^(1)^\top A^(1) * ... * A^(n-1)^\top A^(n-1)
                * A^(n+1)^\top A^(n+1) * ... * A^(N)^\top A^(N)
            A^(n) <- X_(n) (A^(N) \odot ... \odot A^(n+1) \odot A^(n-1) \odot ... \odot A^(1)) V^\dagger
```

```

        normalize columns of A^(n) (store norms in λ)
    end for
until fit stops improving or max iterations reached
return λ, A^(1), ..., A^(N)
end procedure

```

Factors may be initialized randomly or as the R leading left singular vectors of each $X_{(n)}$.

A.5 Tucker: n -mode Product and Flattening

The n -mode product satisfies $G \times_1 A = A G_{(1)}$, and the Tucker model has the flattened forms

$$X_{(1)} \approx A G_{(1)} (C \otimes B)^\top, \quad X_{(2)} \approx B G_{(2)} (C \otimes A)^\top, \quad X_{(3)} \approx C G_{(3)} (B \otimes A)^\top,$$

where $\otimes$ is the Kronecker product (not the Khatri–Rao product). Setting factors to the identity yields the restricted models **Tucker1** ($B = C = I$, so $X_{(1)} = A G_{(1)}$) and **Tucker2** ($C = I$).

A.6 HOSVD: Pseudocode and All-Orthogonality

```

procedure HOSVD(X, R_1, ..., R_N)
    for n = 1, ..., N do
        A^(n) <- R_n leading left singular vectors of X_(n)
    end for
    G <- X ×_1 A^(1)^\top ×_2 ... ×_N A^(N)^\top
    return G, A^(1), ..., A^(N)
end procedure

```

The core of the full HOSVD is *all-orthogonal*: its slices along each mode are mutually orthogonal, which justifies discarding the smallest components when truncating. The truncated HOSVD is fast but is not the best fit in the Frobenius norm.

A.7 HOOI: Derivation and Pseudocode

For columnwise-orthonormal factors $A^{(n)}$ and the optimal core $G = X \times_1 A^{(1)\top} \cdots \times_N A^{(N)\top}$,

$$\begin{aligned} \|X - \llbracket G; A^{(1)}, \dots, A^{(N)} \rrbracket\|^2 &= \|X\|^2 - 2\langle X \times_1 A^{(1)\top} \cdots \times_N A^{(N)\top}, G \rangle + \|G\|^2 \\ &= \|X\|^2 - 2\langle G, G \rangle + \|G\|^2 = \|X\|^2 - \|G\|^2. \end{aligned}$$

Since $\|X\|$ is constant, minimizing the error is equivalent to maximizing

$$\|G\| = \|X \times_1 A^{(1)\top} \cdots \times_N A^{(N)\top}\| = \|A^{(n)\top} W\|, \quad W = X_{(n)} (A^{(N)} \otimes \cdots \otimes A^{(n+1)} \otimes A^{(n-1)} \otimes \cdots \otimes A^{(1)}).$$

The maximizer is obtained from the SVD: $A^{(n)}$ equals the R_n leading left singular vectors of W .

```

procedure HOOI(X, R_1, ..., R_N)
    initialize A^(n) via HOSVD
    repeat
        for n = 1, ..., N do
            W <- X_(n) (A^(N) ⊗ ... ⊗ A^(n+1) ⊗ A^(n-1) ⊗ ... ⊗ A^(1))
            A^(n) <- R_n leading left singular vectors of W
        end for
    until fit stops improving or max iterations reached
    G <- X ×_1 A^(1)^\top ×_2 ... ×_N A^(N)^\top

```

```
    return G, A^(1), ..., A^(N)
end procedure
```